

Relazione POC PON Una Bussola per il futuro

Realizzazione di una pila di Cubetti

Questo primo progetto, pur essendo semplice, è stato fondamentale per prendere confidenza con la programmazione in Python e per approfondire l'uso della libreria *DobotEDU*.

Il robot preleva dei cubetti colorati e crea una pila per fila, per poi eseguire il processo inverso e smontarla.

Questo script è stato riprodotto e testato più volte. È servito non solo per verificare la stabilità dei collegamenti tra il robot e i nuovi IDE esterni, ma anche come vero e proprio test di controllo per convalidare la libreria descritta nel punto successivo, assicurandosi che i comandi venissero recepiti correttamente.

Problemi riscontrati:

- In certe condizioni, nel simulatore, quando ci si vuole spostare alla home salta le alcune funzioni successive, sia su DobotLab che su altri IDE: su questi ultimi basta usare la libreria *time* per far aspettare il robot prima di eseguire altre funzioni.

Libreria che permette di usare sia il Dobot Fisico che il Simulatore

Il progetto nasce dall'esigenza di superare i vincoli imposti dall'ambiente di sviluppo ufficiale DobotLab, offrendo agli sviluppatori la libertà di programmare i bracci robotici Dobot Magician e Magician Lite all'interno del proprio IDE preferito (come VS Code, PyCharm, o tramite script standalone).

Per raggiungere questo scopo, è stata realizzata una libreria in grado di interfacciarsi in modo minimale sia con l'hardware reale sia con il simulatore software. L'architettura è stata progettata per essere trasparente all'utente: lo stesso codice di alto livello può controllare indifferentemente il robot fisico o la sua controparte virtuale, semplicemente variando il parametro di connessione.

Da sottolineare la presenza di **2 librerie una per il Magician Lite (file: *magiciano_lite.py*) e una per il Magician (file: *magician.py*)**.

Il funzionamento della libreria si appoggia sul software **DobotLink**, che deve rimanere costantemente attivo in background.

La connessione viene stabilita tramite **WebSocket**, un protocollo che permette uno scambio di dati bidirezionale e in tempo reale. A seconda del target selezionato, la libreria instrada i comandi verso canali logici differenti:

- **Modalità Simulatore (Virtuale):** La comunicazione viene indirizzata tramite una porta seriale virtuale (**VSP1**), che simula il comportamento del robot direttamente all'interno dell'ambiente software.
- **Modalità Hardware (Fisico):** La comunicazione avviene tramite le porte seriali reali (**Porte COM**), identificando lo specifico indirizzo associato al Dobot Magician o Magician Lite collegato via USB.

Libreria necessarie per il funzionamento:

- websocket
- json
- time
- DobotEDU

Metodi Contenuti nella libreria:

Metodo	Scopo	Parametri
connect (isSumalator)	Avvia la connessione con il robot (fisico o virtuale) tramite DobotLink.	<i>isSimulatore (Bool)</i> : per specificare alla libreria se il robot è fisico o il simulatore
disconnect (isSumalator)	Chiude in modo sicuro la connessione attiva.	<i>isSimulatore (Bool)</i> : per specificare alla libreria se il robot è fisico o il simulatore
setHome ()	per portare alla posizione home il braccio robotico	<i>nessun parametro</i>
move (mode,x,y,z,r)	per spostare il robot in un punto	<i>mode (int)</i> : per specificare il modo con cui si muove il robot <i>x,y,z (int)</i> : sono le coordinate a cui si deve muovere il robot <i>r (int)</i> : di quanto si ruota la ventosa

setVentosa (enable)	per accendere e spegnere la ventosa	<i>enable</i> (bool): se la ventosa deve poter prendere (<i>True</i>) o rilasciare (<i>False</i>)
---------------------	-------------------------------------	---

Problemi riscontrati:

- **Connettersi e disconnettersi al robot fisico:** ho fatto in modo che ci si possa collegare anche senza conoscere la com a cui è collegata al robot.
- **Time:** in certi casi il robot impiega del tempo per prepararsi ad eseguire la funzione successiva per questo ho aggiunto un delay alle funzioni con `time.sleep()`;
- **Non tutte le versioni di python supportano DobotEDU:** io su consiglio dei miei compagni ho usato Python 3.8 e per interpretare il codice ho usato Venv.
- installare la libreria DobotEDU, per aggirare il problema a voi ho creato un file batch (quindi usabile su windows) che dovrebbe installare la libreria usando pip, il file *installerDobotEDU.bat*.

Stampare in 3D con il Dobot Magician

Insieme al mio compagno di squadra, ho cercato di configurare il Dobot per trasformarlo in una stampante 3D il Magician scaricando il software Repetier-Host e facendo un aggiornamento firmware al Magician (per poterlo usare come una stampante 3D).

L'attività si è rivelata complessa e ha richiesto un approccio approfondito al troubleshooting: dopo aver installato l'estrusore sull'end-effector del robot, ho utilizzato il software Repetier-Host per interfacciarmi con la macchina, preparare il file di slicing e tentare la calibrazione e il livellamento del piano di lavoro. Purtroppo, non è stato possibile completare una stampa definitiva a causa di continui problemi tecnici concatenati all'interno dell'ecosistema hardware/software; nel corso del processo si verificavano infatti anomalie ripetute, e non appena riuscivamo a risolvere un problema legato ai movimenti, se ne presentava immediatamente un altro relativo all'estrusione o all'adesione del filamento sul piatto. Successivamente per la frustrazione abbiamo deciso di cambiare progetto.



Problemi risolti:

- Senza **Aggiornamento Firmware tramite DobotStudio**: il motore che dovrebbe mandare il filamento all' estrusore non funziona;
- Senza settare il **protocollo di trasferimento corretto (ASCII)** e la opzione **Comunicazione Ping Pong abilitata** su *impostazione stampante > connessione*;
- Abbiamo dovuto **cambiare le Configurazioni dello Slicer** : impostando le caratteristiche del filo, altezza del piatto e la sua temperatura, dimensione dei layer e temperatura dell' estrusore ...

Problemi irrisolti:

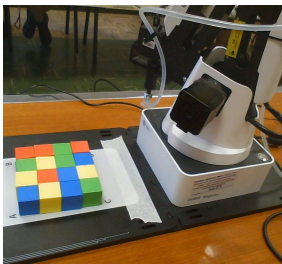
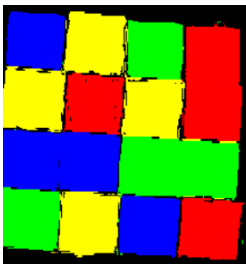
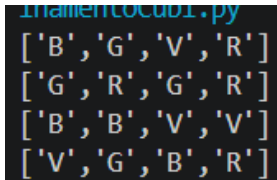
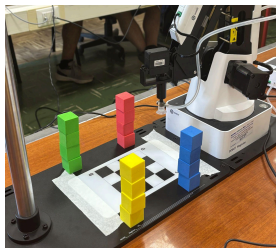
- **in certi casi il magician esegue un Homing** a cui non riesce ad arrivare per via dei limiti dei suoi assi, **in certi casi si risolve cambiando le impostazioni di Home X, Y e Z** in *impostazioni stampante > opzioni piano di stampa*, anche se si può **ripresentare** il problema;
- **Con qualsiasi protocollo di trasferimento** il robot non segue il percorsi di stampa **si limita a seguire delle linee diagonali** apparentemente parallele senza stampare nulla e con nessuna corrispondenza con ciò che appare nell' ambiente virtuale che dovrebbe mostrare ciò che viene stampato.

E' da notare comunque che con queste modifiche non siamo riusciti a stampare nulla.

Kit Vision Riconoscimento colore dei Cubetti

L' obiettivo è stato quello di riconoscere i colori di alcuni cubetti disposti in ordine casuale in un quadrato 4x4 con una fotocamera, una volta riconosciuti i colori viene mandata al robot Magician una matrice contenente dei caratteri identificativi per ogni colore (R per il rosso, G per il giallo, B per il blu, V per il verde e '?' se il colore non rientrava in un range) per poter realizzare una pila per ogni colore. Per utilizzare la fotocamera è stata utilizzata la libreria OpenCV mentre per la manovrazione del robot abbiamo utilizzato la mia libreria fatta in precedenza.

Fasi risolutive:

Cubi disordinati	ROI	Matrice	Pile
			

Per mantenere il codice più pulito ho creato un package contenente le funzioni per ottenere la matrice di colori (file: *domatrice.py*).

Metodo	Scopo	Parametri
<i>cattura_e_ritaglia_roi</i> (<i>sorgente_camera</i>)	Permette di ottenere una ROI (una sezione di immagine, che può essere divisa anche in più sezioni, nel nostro caso 4x4)	<u><i>sorgente_camera</i></u> (<i>int</i>): l' indice che l' SO assegna alla fotocamera (parte da 0)

<i>analizza_colori_roi_hsv</i> (<i>roi_totale</i>)	data la ROI calcola il colore medio di ogni cella e in in elemento di una matrice corrispondente ne inserisce un carattere corrispondente. alla fine ritorna la matrice	roi_totale (ROI): la roi da cui vengono calcolati i colori medi per ogni cella
---	--	--

Problemi Ricontrati:

- **La camera del kit funzionava solo con il suo software** (*Machine Vision Software*, o *MVS*): quindi abbiamo optato per utilizzarne un' altra che poteva essere usata con OpenCV, poiché letta dal PC come una fotocamera indicizzata;
- Non sapevamo come ottenere un **range per ogni colore**: per questo li abbiamo chiesti a Gemini;
- Il Magician sembrava **cambiare sistema di coordinate ogni volta che veniva acceso**: per questo abbiamo optato di eseguire un homing prima di farlo muovere per resettare il sistema di riferimento.

